FINAL PROGRESS REPORT

Week 5 | March 2026 **Course:** One Credit Independent Study | **Student:** Aavash Kuikel

Project Overview

- **Project Title:** Extending AI-Generated Source Code Detection to Next-Generation LLMs
- **GitHub:** github.com/akuikel/detectingAIGeneratedCode
- **Advisor:** Dr. Zhang

> ★ **MAJOR FINDING — Counter-Intuitive Result**
>
> Newer-generation LLMs (Claude Haiku, GPT-4o, Gemini 2.5 Flash) are significantly EASIER to detect as AI-generated, not harder. Our best classifier achieves F1 = 0.9939 on Gemini 2.5 — an improvement of +16.8 percentage points over the original paper's best of 0.8255 on ChatGPT. This suggests that as LLMs become more capable, they produce more structurally uniform and stylistically consistent code, creating a stronger and more recognizable "fingerprint" for automated detectors.

Overview

Week 5 marks the completion of the project. The classifier was run, results were analyzed and compared against the original paper's baselines, the full codebase was pushed to GitHub with documentation, and the project was prepared for demonstration.

Results — Classifier Performance

The following table shows F1 scores for each new LLM across three embedding types using 5-fold stratified cross-validation with logistic regression:

Model	Embedding	F1	Human F1	AI F1

ChatGPT (paper)	Combined	0.8255	0.8369	0.8141
GPT-4 (paper)	Combined	—	—	—
Gemini Pro (paper)	Combined	—	—	—
Claude Haiku (ours)	Code	0.9695	0.9681	0.9710
Claude Haiku (ours)	AST	0.9817	0.9818	0.9816
Claude Haiku (ours)	Combined	0.9756	0.9745	0.9768
GPT-4o (ours)	Code	0.9787	0.9781	0.9793
GPT-4o (ours)	AST	0.9635	0.9629	0.9641
GPT-4o (ours)	Combined	0.9879	0.9874	0.9883

Gemini 2.5 (ours)	Code	0.9847	0.9848	0.9847
Gemini 2.5 (ours)	AST	0.9818	0.9820	0.9815
Gemini 2.5 (ours)	Combined	0.9939	0.9939	0.9939

(Table Source:)

Key Finding:

- Newer LLMs (Claude, GPT-4o, Gemini 2.5) are significantly easier to detect than older models tested in the original paper.
- The best F1 of 0.9939 (Gemini 2.5, combined embedding) represents an improvement of +16.8 percentage points over the paper's best of 0.8255.
- While counter-intuitive, results suggest that newer models write more structurally consistent and stylistically uniform code than their predecessors, making them highly distinguishable from the variable, idiosyncratic patterns of human-written code.

-

> **Why Are Newer Models Easier to Detect?**
>
> Modern LLMs like GPT-4o and Gemini 2.5 are trained with RLHF (Reinforcement Learning from Human Feedback) on far larger datasets, causing them to produce "textbook-perfect" code — consistently structured, predictably commented, uniformly indented. Human code is messier: developers take shortcuts, use unusual patterns, leave inconsistencies. This stylistic uniformity of newer AI-generated code creates a stronger, more detectable structural fingerprint that CodeBERT embeddings capture very effectively.

-

Week 5 Activities

Classification and Analysis:

- Ran logistic regression classifier with 5-fold stratified cross-validation on all 3 models x 3 embedding types.
- Computed F1, Human F1, AI F1, Accuracy, TPR, and TNR for each configuration.
- Compared results against paper baseline (ChatGPT combined embedding F1 = 0.8255).
- Identified key finding: newer LLMs show higher detectability because they write more formulaic, structurally consistent code.

GitHub and Documentation:

- Pushed all 6 pipeline scripts to github.com/akuikel/detectingAIGeneratedCode.
- Pushed all dataset files: raw/, merged/, and ast_processed/ directories.
- Pushed final results CSV: results/final_results.csv.
- Wrote a comprehensive README with results table, methodology, and step-by-step reproduction instructions.
- Added .gitignore to exclude large embedding files, split directories, and API keys.
- Ensured all API keys were removed from committed code before the push.

Preparation for Demo:

- Verified that running python runClassifier.py reproduces all results from saved splits.
- Confirmed the full pipeline can be re-run from scratch with a single sequence of 5 commands.
- Documented all version-specific issues and their fixes for reproducibility.

Methodology Summary

What Was Done:

- **Dataset:** All 164 Python problems from HumanEval-X benchmark (same as original paper).

- **Code generation:** Each of 3 LLMs prompted with function signature + docstring, asked to return only the complete function.
- **Data labeling:** AI-generated code labeled 0, human canonical solutions labeled 1; 328 samples per model.
- **AST generation:** tree-sitter parser with variable renaming normalization.
- **Embeddings:** CodeBERT (microsoft/codebert-base) CLS token, 768-dim; three types: code, AST, combined (1536-dim).
- **Classifier:** Logistic regression with StandardScaler, 5-fold stratified cross-validation.

Difference from Original Paper:

- The original used CodeT5+ 110M for embeddings; this extension uses CodeBERT due to Python 3.13 compatibility.
- CodeBERT is the same model used in GPTSniffer (the paper's prior state-of-the-art baseline), so comparison remains valid.
- The original tested 4 LLMs from 2023-2024; this extension tests 3 next-generation successors from 2025-2026.

Conclusion

This project successfully extended the ICSE 2025 study by evaluating three next-generation LLMs on the AI-generated code detection task. The pipeline was built from scratch, resolving significant compatibility issues along the way. The results reveal a surprising and academically interesting finding: state-of-the-art LLMs in 2025-2026 produce code that is more easily detected as AI-generated than their 2023-era predecessors.

All code, data, and results are publicly available at:
github.com/akuikel/detectingAIGeneratedCode